

R-Code Beispiele aus WDDA Vorlesung 5 (Kapitel 5)

WDDA – Datenmanagement und Datenanalyse

Prof. Dr. Ulrich Matter

2026-05-06

Inhaltsverzeichnis

1	Einleitung	1
2	Vorbereitung	2
3	Variation	2
4	Erklärende Variable	4
5	Kandidaten vergleichen	5
6	Abweichungen vom Modell: Quantifizierung	6
7	Best Fit: Schätzgerade	8
8	Residuen	11
9	TSS und RSS	14
10	Zerlegung der Variation	15
11	Bestimmtheitsmass	17
12	Erwartungen an Residuen	19
13	Prognose	22
14	Zusammenfassung	24

1 Einleitung

Dieses Dokument fasst alle R-Code-Beispiele der Folien aus der WDDA FS2026 Vorlesung 5 (Kapitel 5) zusammen. Der Fokus liegt auf der einfachen linearen Regression, einem grundlegenden statistischen Verfahren zur Modellierung des Zusammenhangs zwischen zwei Variablen.

Die einfache lineare Regression ist ein statistisches Verfahren, das den linearen Zusammenhang zwischen einer abhängigen Variable (Zielvariable) und einer unabhängigen Variable (Prädiktor) modelliert. Das Ziel ist es, eine Gerade zu finden, die den Zusammenhang zwischen den beiden Variablen am besten beschreibt.

Zentrale Konzepte in diesem Bereich sind:

1. **Variation und Kovarianz:** Wie stark variieren die Variablen und wie hängen sie zusammen?

2. **Methode der kleinsten Quadrate:** Wie findet man die “beste” Regressionsgerade?
3. **Residuen:** Wie gross sind die Abweichungen zwischen den beobachteten und den vorhergesagten Werten?
4. **Bestimmtheitsmass (R^2):** Wie gut erklärt das Modell die Variation in den Daten?
5. **Prognose:** Wie können wir das Modell für Vorhersagen nutzen?

2 Vorbereitung

```
# Pakete laden
library(readxl)

# Daten laden
data <- read_excel("data/WDDA_05.xlsx", sheet = "Advertising")

# variablen direkt anwählbar machen
attach(data)

## tibble [200 x 4] (S3: tbl_df/tbl/data.frame)
##   $ TV          : num [1:200] 230.1 44.5 17.2 151.5 180.8 ...
##   $ radio       : num [1:200] 37.8 39.3 45.9 41.3 10.8 48.9 32.8 19.6 2.1 2.6 ...
##   $ newspaper: num [1:200] 69.2 45.1 69.3 58.5 58.4 75 23.5 11.6 1 21.2 ...
##   $ sales      : num [1:200] 22.1 10.4 9.3 18.5 12.9 7.2 11.8 13.2 4.8 10.6 ...
```

R-Erklärung:

- Die Funktion `library()` lädt ein installiertes Paket in die aktuelle R-Sitzung.
- Mit `read_excel()` aus dem Paket `readxl` können Excel-Dateien eingelesen werden. Der Parameter `sheet` gibt an, welches Tabellenblatt verwendet werden soll.
- Die Funktion `attach()` macht die Variablen eines Dataframes direkt ansprechbar, ohne dass man den Dataframe-Namen voranstellen muss (z.B. `sales` statt `data$sales`).
- Die Funktion `str()` zeigt die Struktur eines Objekts, einschliesslich Datentypen und Dimensionen.

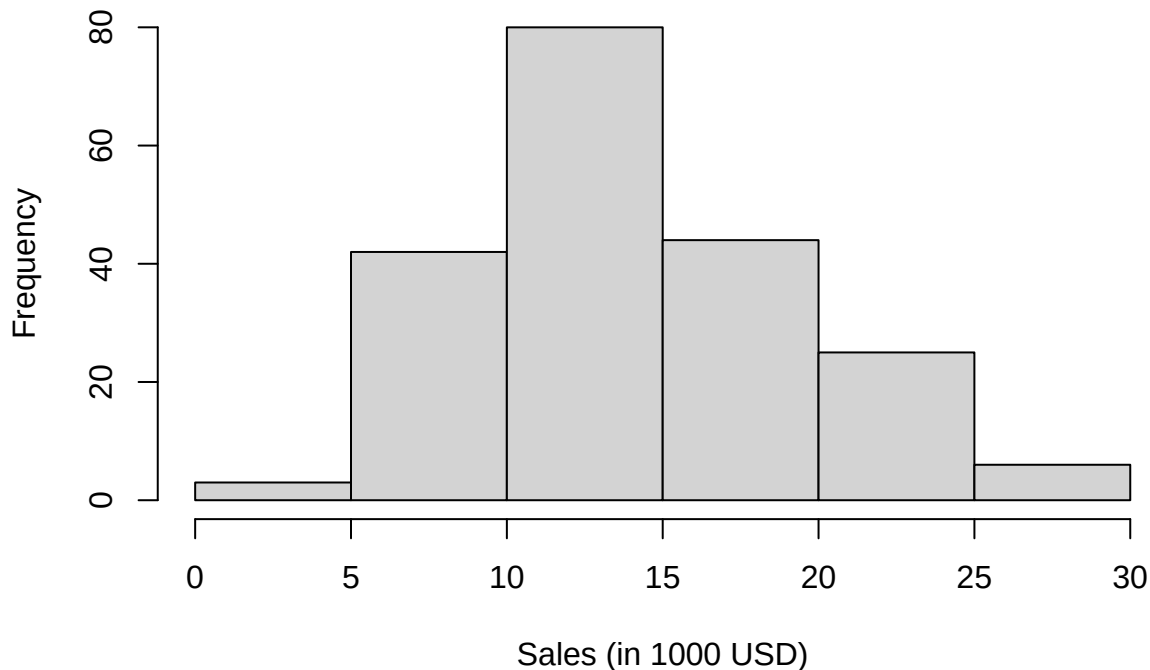
Hinweis zu Datenpfaden: In R-Markdown-Dateien werden relative Pfade vom Speicherort der Datei aus interpretiert, daher der Pfad `../data/WDDA_05.xlsx`. In regulären R-Skripten würde der Pfad relativ zum Arbeitsverzeichnis angegeben werden.

3 Variation

Bevor wir mit der Regression beginnen, betrachten wir die Variation in unserer Zielvariable (`sales`).

```
# Basis-Visualisierung: Zielvariable
hist(sales, main = "Verteilung der Verkaufszahlen", xlab = "Sales (in 1000 USD)")
```

Verteilung der Verkaufszahlen



```
cat("Varianz von sales:", round(var(sales), 2), "\n")
```

```
## Varianz von sales: 27.22
```

```
# Zusätzliche Statistiken
```

```
cat("Mittelwert von sales:", round(mean(sales), 2), "\n")
```

```
## Mittelwert von sales: 14.02
```

```
cat("Standardabweichung von sales:", round(sd(sales), 2), "\n")
```

```
## Standardabweichung von sales: 5.22
```

```
cat("Minimum von sales:", min(sales), "\n")
```

```
## Minimum von sales: 1.6
```

```
cat("Maximum von sales:", max(sales), "\n")
```

```
## Maximum von sales: 27
```

Statistische Erklärung:

- Die **Varianz** ist ein Mass für die Streuung der Daten um ihren Mittelwert.
- Eine hohe Varianz bedeutet, dass die Werte weit um den Mittelwert streuen.
- Die Varianz wird berechnet als der Durchschnitt der quadrierten Abweichungen vom Mittelwert: $\text{Var}(X) = \frac{\sum (x_i - \bar{x})^2}{n}$
- Die **Standardabweichung** ist die Quadratwurzel der Varianz und hat den Vorteil, dass sie in der

gleichen Einheit wie die Daten selbst gemessen wird.

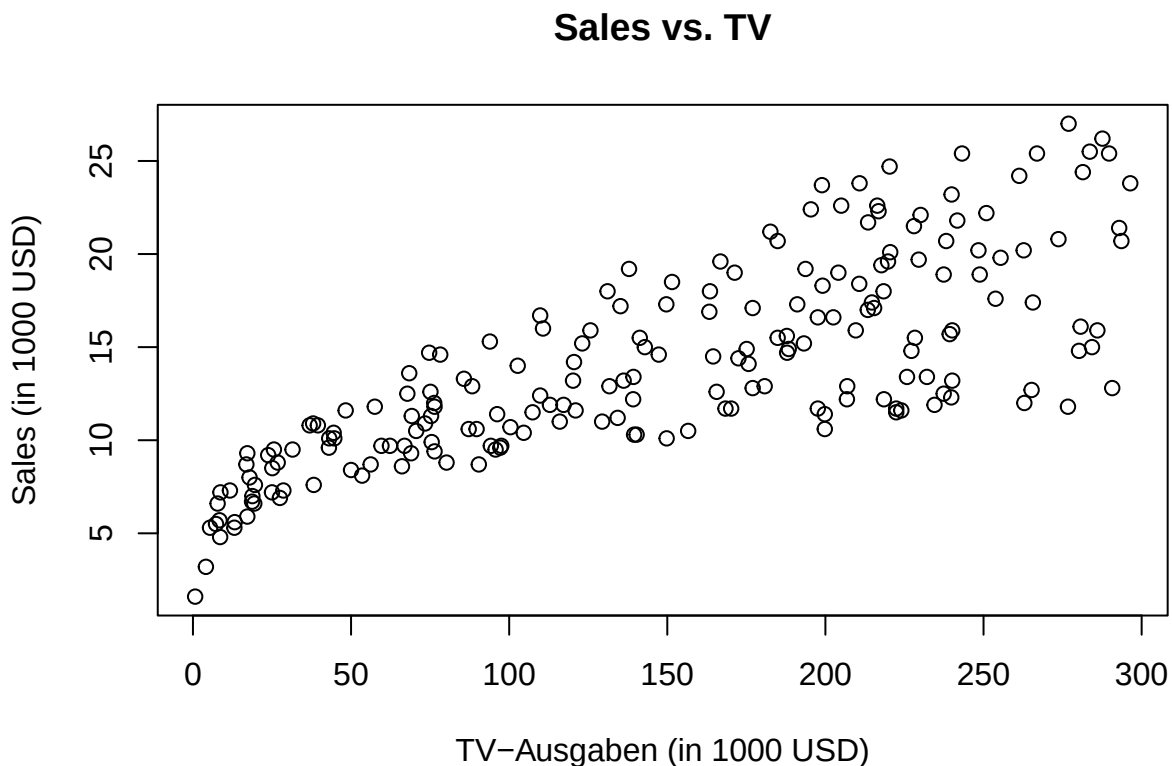
R-Erklärung:

- Die Funktion `hist()` erstellt ein Histogramm, das die Häufigkeitsverteilung der Daten zeigt.
- Die Funktion `var()` berechnet die Varianz eines Vektors.
- Die Funktion `mean()` berechnet den Mittelwert eines Vektors.
- Die Funktion `sd()` berechnet die Standardabweichung eines Vektors.
- Die Funktionen `min()` und `max()` geben den kleinsten bzw. grössten Wert eines Vektors zurück.

4 Erklärende Variable

Nun betrachten wir den Zusammenhang zwischen unserer Zielvariable (sales) und der erklärenden Variable (TV-Werbeausgaben).

```
# Scatterplot: Zusammenhang zwischen TV und Verkaufszahlen
plot(sales ~ TV, main = "Sales vs. TV", xlab = "TV-Ausgaben (in 1000 USD)", ylab = "Sales
  ↪ (in 1000 USD)")
```



```
# Korrelation berechnen
korrelation <- cor(sales, TV)
cat("Korrelation zwischen sales und TV:", round(korrelation, 3), "\n")
```

```
## Korrelation zwischen sales und TV: 0.782
```

```
# Kovarianz berechnen
kovarianz <- cov(sales, TV)
cat("Kovarianz zwischen sales und TV:", round(kovarianz, 3), "\n")
```

Kovarianz zwischen sales und TV: 350.39

Statistische Erklärung:

- Ein **Streudiagramm** (Scatterplot) zeigt den Zusammenhang zwischen zwei kontinuierlichen Variablen.
- Jeder Punkt repräsentiert eine Beobachtung mit den entsprechenden Werten für beide Variablen.
- Die **Korrelation** ist ein standardisiertes Mass für den linearen Zusammenhang zwischen zwei Variablen und liegt zwischen -1 und +1.
 - Eine Korrelation nahe +1 bedeutet einen starken positiven linearen Zusammenhang.
 - Eine Korrelation nahe -1 bedeutet einen starken negativen linearen Zusammenhang.
 - Eine Korrelation nahe 0 bedeutet keinen linearen Zusammenhang.
- Die **Kovarianz** ist ein unstandardisiertes Mass für den linearen Zusammenhang zwischen zwei Variablen.
 - Die Kovarianz hat keine feste Skala und hängt von den Einheiten der Variablen ab.
 - Eine positive Kovarianz bedeutet, dass die Variablen tendenziell gemeinsam steigen oder fallen.
 - Eine negative Kovarianz bedeutet, dass eine Variable tendenziell steigt, wenn die andere fällt.

R-Erklärung:

- Die Formel `sales ~ TV` in `plot()` gibt an, dass `sales` auf der y-Achse und `TV` auf der x-Achse dargestellt werden soll.
- Die Funktion `cor()` berechnet die Pearson-Korrelation zwischen zwei Vektoren.
- Die Funktion `cov()` berechnet die Kovarianz zwischen zwei Vektoren.

5 Kandidaten vergleichen

Wir vergleichen verschiedene mögliche Regressionsgeraden, um zu sehen, welche den Zusammenhang zwischen TV-Werbeausgaben und Verkaufszahlen am besten beschreibt.

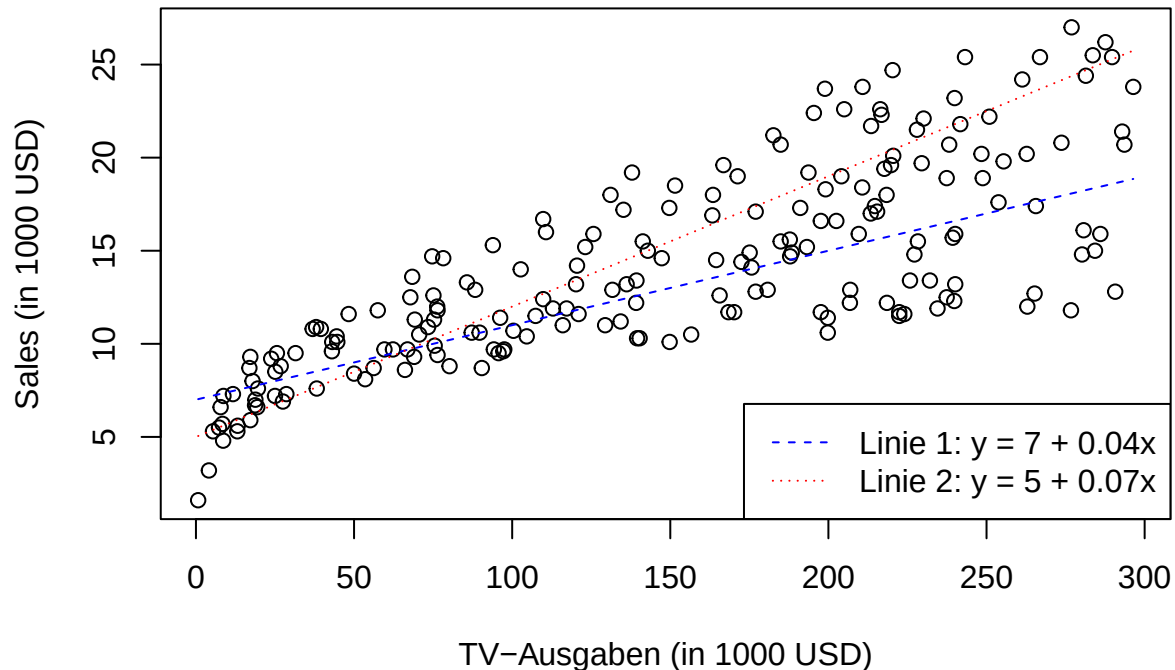
```
# Scatterplot erstellen
plot(sales ~ TV, main = "Sales vs. TV - Vergleich von Regressionsgeraden",
      xlab = "TV-Ausgaben (in 1000 USD)", ylab = "Sales (in 1000 USD)")

# Zwei mögliche Linien (manuell definiert)
line1 <- function(x) { 7 + 0.04 * x }
line2 <- function(x) { 5 + 0.07 * x }

# Linien zum Plot hinzufügen
curve(line1, add = TRUE, col = "blue", lty = 2)
curve(line2, add = TRUE, col = "red", lty = 3)

# Legende hinzufügen
legend("bottomright",
      legend = c("Linie 1: y = 7 + 0.04x", "Linie 2: y = 5 + 0.07x"),
      col = c("blue", "red"),
      lty = c(2, 3))
```

Sales vs. TV – Vergleich von Regressionsgeraden



Statistische Erklärung:

- Eine **Regressionsgerade** hat die Form $y = b + b x$, wobei:
 - b ist der y-Achsenabschnitt (Intercept): Der vorhergesagte Wert von y , wenn $x = 0$ ist.
 - b ist die Steigung (Slope): Die Änderung in y , die mit einer Einheit Änderung in x verbunden ist.
- Verschiedene Geraden können den Zusammenhang zwischen zwei Variablen unterschiedlich gut beschreiben.
- Die “beste” Gerade ist diejenige, die die Abweichungen zwischen den beobachteten und den vorhergesagten Werten minimiert.

R-Erklärung:

- Die Funktion `curve()` mit dem Parameter `add = TRUE` fügt eine Kurve zu einem bestehenden Plot hinzu.
- Der Parameter `lty` (line type) in `curve()` und `legend()` gibt den Linientyp an (1 = durchgezogen, 2 = gestrichelt, 3 = gepunktet, etc.).
- Die Funktion `legend()` fügt eine Legende zum Plot hinzu.

6 Abweichungen vom Modell: Quantifizierung

Um zu bestimmen, welche Regressionsgerade besser ist, quantifizieren wir die Abweichungen zwischen den beobachteten und den vorhergesagten Werten.

```
# Vorhergesagte Werte für beide Linien berechnen
pred_line1 <- line1(TV)
pred_line2 <- line2(TV)

# Residuen (Abweichungen) berechnen
resid_line1 <- sales - pred_line1
```

```

resid_line2 <- sales - pred_line2

# Quadrierte Residuen berechnen
sq_resid_line1 <- resid_line1^2
sq_resid_line2 <- resid_line2^2

# Summe der quadrierten Residuen (RSS) berechnen
rss_line1 <- sum(sq_resid_line1)
rss_line2 <- sum(sq_resid_line2)

# Ergebnisse ausgeben
cat("Sum of Squared Errors (RSS):\n")

## Sum of Squared Errors (RSS):

cat("Linie 1 (y = 7 + 0.04x):", round(rss_line1, 2), "\n")

## Linie 1 (y = 7 + 0.04x): 2446.13

cat("Linie 2 (y = 5 + 0.07x):", round(rss_line2, 2), "\n")

## Linie 2 (y = 5 + 0.07x): 3165.51

# Visualisierung der Residuen für die erste Beobachtung
plot(sales ~ TV, main = "Residuen für die erste Beobachtung",
     xlab = "TV-Ausgaben (in 1000 USD)", ylab = "Sales (in 1000 USD)")
curve(line1, add = TRUE, col = "blue", lty = 2)
curve(line2, add = TRUE, col = "red", lty = 3)

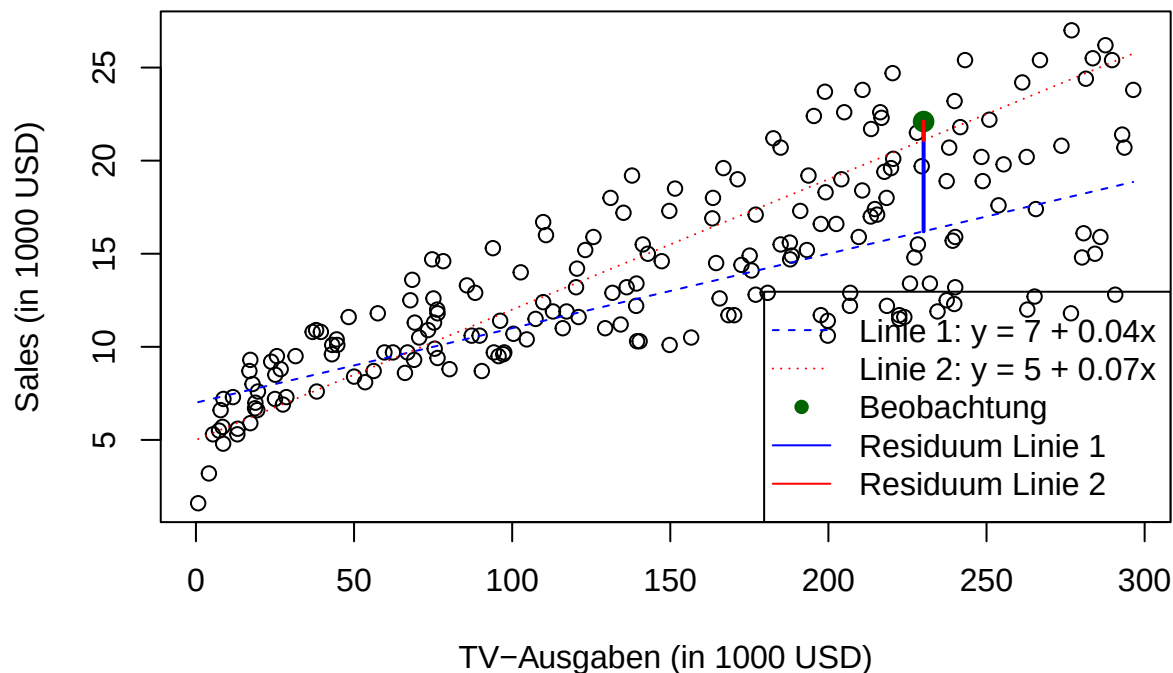
# Erste Beobachtung hervorheben
points(TV[1], sales[1], col = "darkgreen", pch = 16, cex = 1.5)

# Residuen für die erste Beobachtung zeichnen
segments(TV[1], sales[1], TV[1], pred_line1[1], col = "blue", lwd = 2)
segments(TV[1], sales[1], TV[1], pred_line2[1], col = "red", lwd = 2)

# Legende hinzufügen
legend("bottomright",
      legend = c("Linie 1: y = 7 + 0.04x", "Linie 2: y = 5 + 0.07x", "Beobachtung",
        ↪ "Residuum Linie 1", "Residuum Linie 2"),
      col = c("blue", "red", "darkgreen", "blue", "red"),
      lty = c(2, 3, NA, 1, 1),
      pch = c(NA, NA, 16, NA, NA))

```

Residuen für die erste Beobachtung



Statistische Erklärung:

- **Residuen** sind die Differenzen zwischen den beobachteten und den vorhergesagten Werten: $e_i = y_i - \hat{y}_i$
- Die **Summe der quadrierten Residuen** (Residual Sum of Squares, RSS) ist ein Mass für die Gesamtabweichung des Modells: $RSS = \sum (y_i - \hat{y}_i)^2$
- Je kleiner die RSS, desto besser passt das Modell zu den Daten.
- Die **Methode der kleinsten Quadrate** (Least Squares) wählt die Regressionsgerade, die die RSS minimiert.
- In diesem Beispiel hat Linie 1 ($y = 7 + 0.04x$) eine kleinere RSS als Linie 2 ($y = 5 + 0.07x$), was darauf hindeutet, dass sie besser zu den Daten passt.

R-Erklärung:

- Der Operator `^2` berechnet das Quadrat eines Vektors.
- Die Funktion `sum()` berechnet die Summe der Elemente eines Vektors.
- Die Funktion `segments()` zeichnet Liniensegmente zwischen zwei Punkten.
- Der Parameter `pch` (plotting character) in `points()` und `legend()` gibt das Symbol für Punkte an.
- Der Parameter `cex` (character expansion) in `points()` gibt die Grösse der Punkte an.

7 Best Fit: Schätzgerade

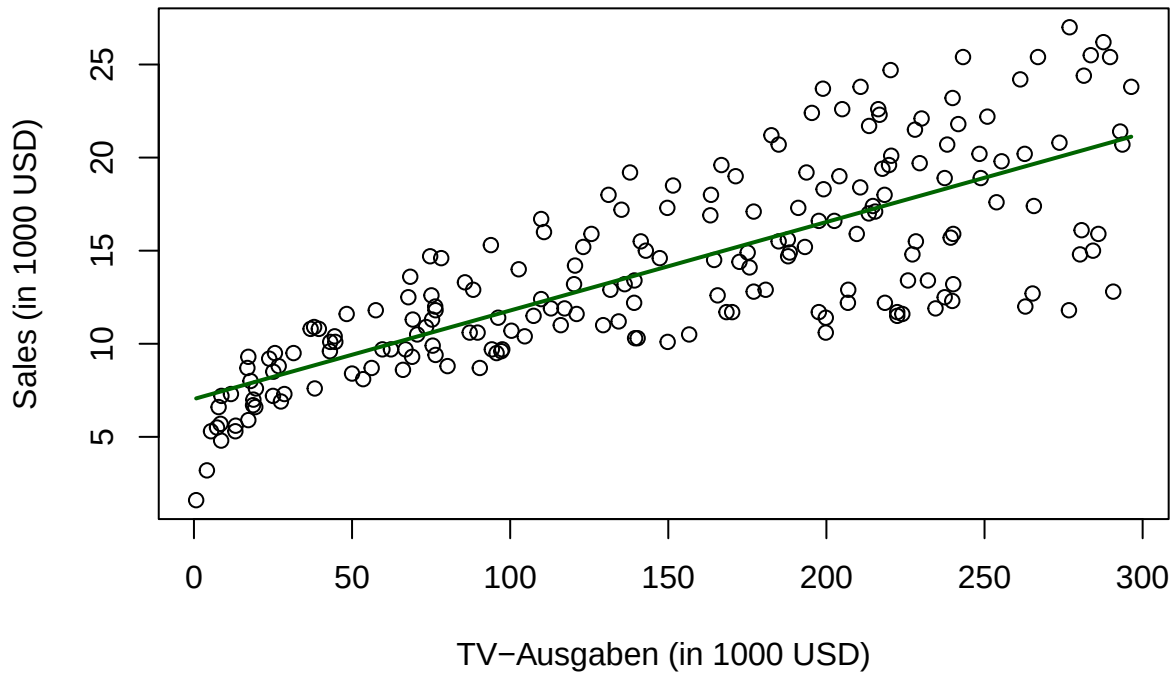
Nun berechnen wir die optimale Regressionsgerade nach der Methode der kleinsten Quadrate.

```
# Methode der kleinsten Quadrate (händisch)
b1 <- cor(sales, TV) * sd(sales) / sd(TV)
b0 <- mean(sales) - b1 * mean(TV)
lbf <- function(x) { b0 + b1 * x }
```



```
# Zeichne Schätzgerade
plot(sales ~ TV, main = "Beste Gerade (Least Squares)",
     xlab = "TV-Ausgaben (in 1000 USD)", ylab = "Sales (in 1000 USD)")
curve(lbf(x), add = TRUE, col = "darkgreen", lwd = 2)
```

Beste Gerade (Least Squares)



```
# Abweichung (RSS) berechnen
pred_lbf <- lbf(TV)
resid_lbf <- sales - pred_lbf
rss <- sum(resid_lbf^2)
cat("Koeffizienten der Schätzgerade:\n")
```

```
## Koeffizienten der Schätzgerade:
```

```
cat("Achsenabschnitt (b):", round(b0, 4), "\n")
```

```
## Achsenabschnitt (b): 7.0326
```

```
cat("Steigung (b):", round(b1, 4), "\n")
```

```
## Steigung (b): 0.0475
```

```
cat("Minimale Abweichung (RSS) der Schätzgerade:", round(rss, 2), "\n")
```

```
## Minimale Abweichung (RSS) der Schätzgerade: 2102.53
```

```
# Regressionsmodell mit lm()
```

```
mdl <- lm(sales ~ TV)
```

```
summary(mdl)
```

```
##
```

```
## Call:
```

```
## lm(formula = sales ~ TV)
```

```
##
```

```
## Residuals:
```

```
##      Min       1Q   Median       3Q      Max
## -8.3860 -1.9545 -0.1913  2.0671  7.2124
```

```
##
```

```
## Coefficients:
```

```
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  7.032594    0.457843   15.36  <2e-16 ***
## TV           0.047537    0.002691   17.67  <2e-16 ***
```

```
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
```

```
## Residual standard error: 3.259 on 198 degrees of freedom
```

```
## Multiple R-squared:  0.6119, Adjusted R-squared:  0.6099
```

```
## F-statistic: 312.1 on 1 and 198 DF, p-value: < 2.2e-16
```

```
# Vergleich der händischen Berechnung mit lm()
```

```
cat("\nVergleich der Koeffizienten:\n")
```

```
##
```

```
## Vergleich der Koeffizienten:
```

```
cat("Händisch: y =", round(b0, 4), "+", round(b1, 4), "* x\n")
```

```
## Händisch: y = 7.0326 + 0.0475 * x
```

```
cat("Mit lm(): y =", round(coef(mdl)[1], 4), "+", round(coef(mdl)[2], 4), "* x\n")
```

```
## Mit lm(): y = 7.0326 + 0.0475 * x
```

```
# Beide Geraden im Plot darstellen
```

```
plot(sales ~ TV, main = "Vergleich: Händische Berechnung vs. lm()",
```

```
      xlab = "TV-Ausgaben (in 1000 USD)", ylab = "Sales (in 1000 USD)")
```

```
curve(lbf(x), add = TRUE, col = "darkgreen", lwd = 2)
```

```
abline(mdl, col = "orange", lwd = 2, lty = 2)
```

```
legend("bottomright",
```

```
      legend = c(sprintf("Händisch: y = %.4f + %.4fx", b0, b1),
```

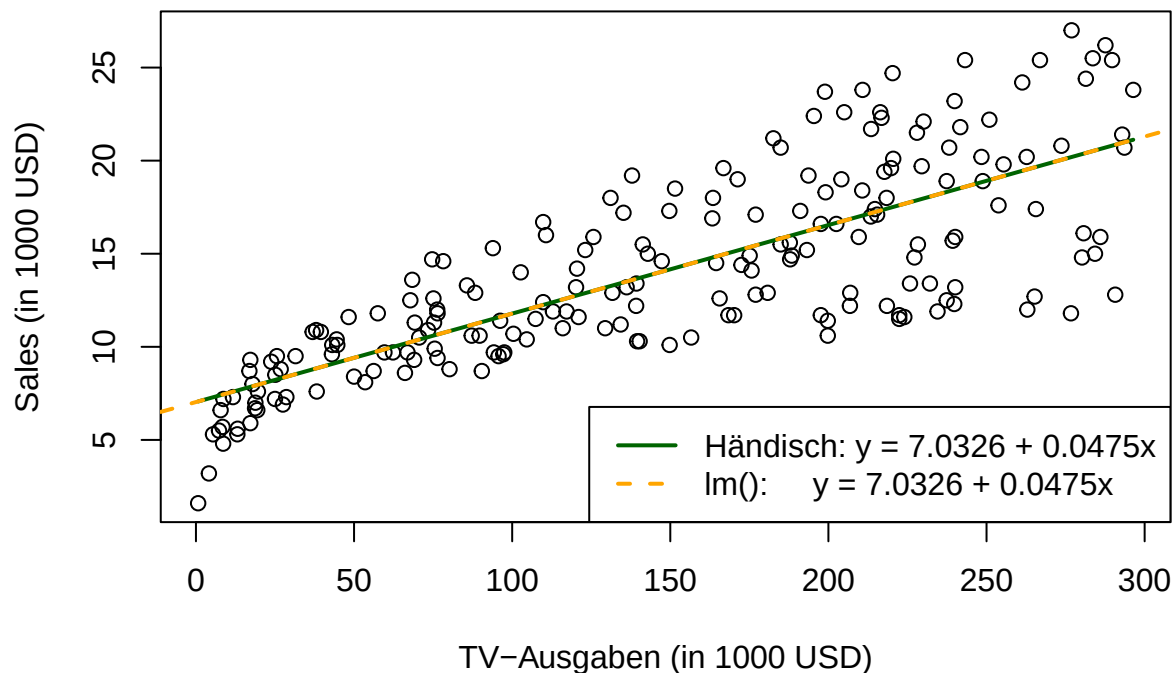
```
                  sprintf("lm(): y = %.4f + %.4fx", coef(mdl)[1], coef(mdl)[2])),
```

```
      col = c("darkgreen", "orange"),
```

```
      lty = c(1, 2),
```

```
      lwd = 2)
```

Vergleich: Händische Berechnung vs. lm()



Statistische Erklärung:

- Die **Methode der kleinsten Quadrate** liefert folgende Formeln für die Koeffizienten:
 - $b = \text{Cov}(X, Y) / \text{Var}(X) = r * (s_Y / s_X)$, wobei r die Korrelation zwischen X und Y ist.
 - $b = Y - b * X$
- Diese Formeln garantieren, dass die resultierende Gerade die Summe der quadrierten Residuen (RSS) minimiert.
- Die Funktion `lm()` in R implementiert die Methode der kleinsten Quadrate und liefert zusätzliche Informationen über das Regressionsmodell.
- Die händische Berechnung und `lm()` sollten zu identischen Koeffizienten führen.

R-Erklärung:

- Die Funktion `lm()` (linear model) passt ein lineares Regressionsmodell an die Daten an.
- Die Formel `sales ~ TV` in `lm()` gibt an, dass `sales` die abhängige Variable und `TV` die unabhängige Variable ist.
- Die Funktion `summary()` liefert eine detaillierte Zusammenfassung des Regressionsmodells, einschliesslich Koeffizienten, Standardfehler, t-Werte, p-Werte und R^2 .
- Die Funktion `coef()` extrahiert die Koeffizienten aus einem Regressionsmodell.
- Die Funktion `abline()` fügt eine Gerade zu einem bestehenden Plot hinzu, basierend auf einem Regressionsmodell.

8 Residuen

Wir betrachten nun die Residuen des Regressionsmodells, um die Qualität der Anpassung zu beurteilen.

```
# 1. Residuum (Beobachtung: TV = 230.1, sales = 22.1)
tv_val <- 230.1
sales_obs <- 22.1
```

```
# Vorhergesagter Wert berechnen
sales_pred <- b0 + b1 * tv_val
cat("Vorhergesagter Wert bei TV =", tv_val, ":", round(sales_pred, 2), "\n")
```

```
## Vorhergesagter Wert bei TV = 230.1 : 17.97
```

```
# Residuum berechnen
resid1 <- sales_obs - sales_pred
cat("1. Residuum:", round(resid1, 2), "\n")
```

```
## 1. Residuum: 4.13
```

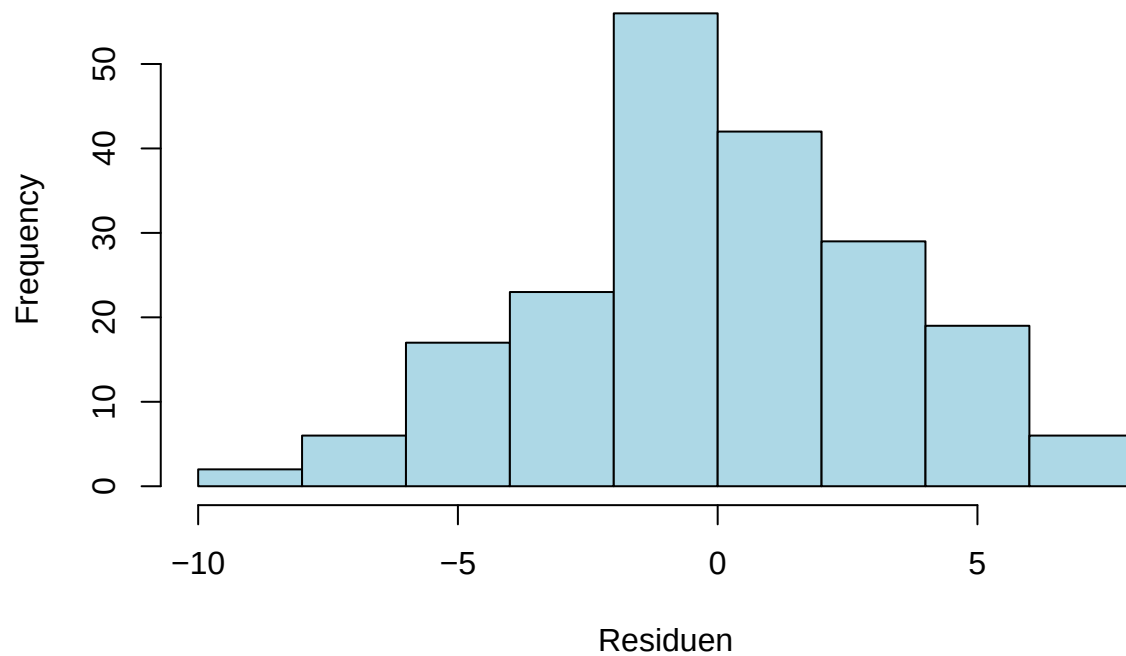
```
# Alternative Berechnung mit predict()
sales_hat <- predict(md1, newdata = data.frame(TV = tv_val))
resid1_alt <- sales_obs - sales_hat
cat("1. Residuum (mit predict()):", round(resid1_alt, 2), "\n")
```

```
## 1. Residuum (mit predict()): 4.13
```

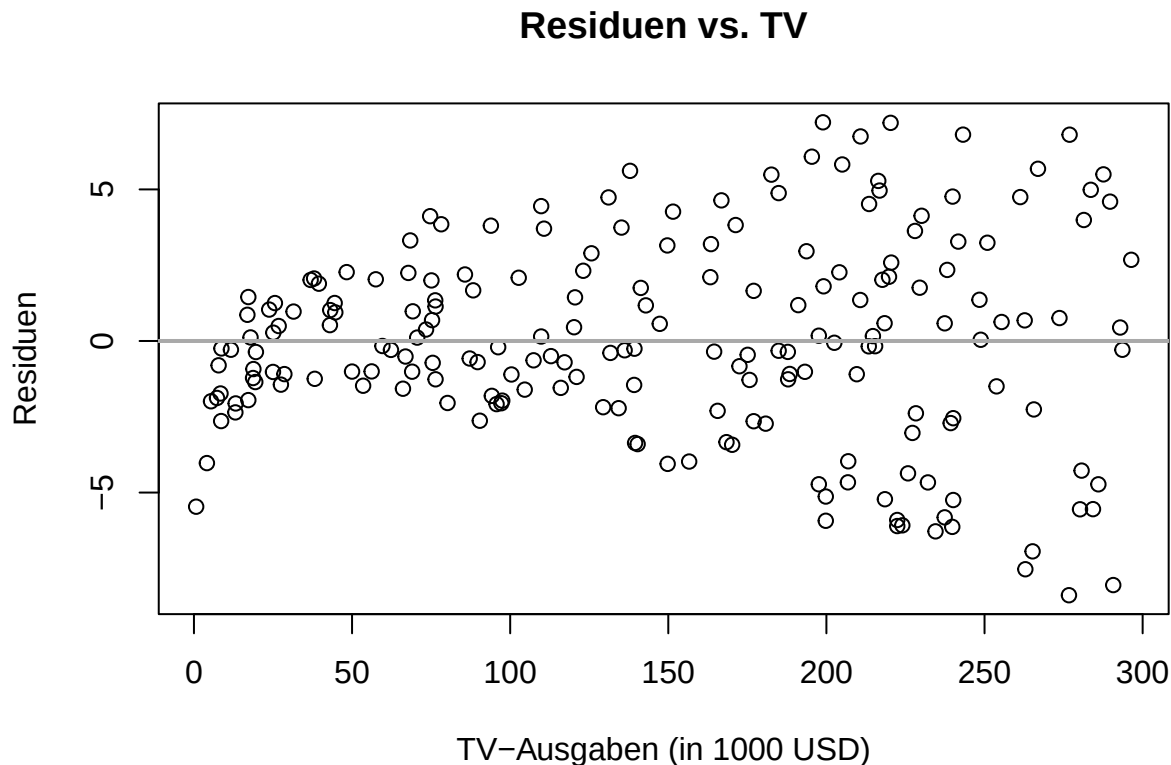
```
# Alle Residuen berechnen
residuen <- resid(md1)
```

```
# Histogramm der Residuen
hist(residuen,
      main = "Histogramm der Residuen",
      xlab = "Residuen",
      col = "lightblue")
```

Histogramm der Residuen



```
# Residuen vs. Prädiktor
plot(residuen ~ TV,
     main = "Residuen vs. TV",
     xlab = "TV-Ausgaben (in 1000 USD)",
     ylab = "Residuen")
abline(h = 0, col = "darkgray", lwd = 2)
```



Statistische Erklärung:

- **Residuen** sind die Differenzen zwischen den beobachteten und den vorhergesagten Werten: $e_i = y_i - \hat{y}_i$
- Die Residuen geben Aufschluss über die Qualität der Anpassung des Modells:
 - Wenn die Residuen zufällig um Null verteilt sind, ist das Modell angemessen.
 - Wenn die Residuen Muster aufweisen, könnte das Modell verbessert werden.
- Die Analyse der Residuen ist ein wichtiger Schritt bei der Überprüfung der Annahmen der linearen Regression:
 - Normalität: Die Residuen sollten normalverteilt sein.
 - Homoskedastizität: Die Varianz der Residuen sollte konstant sein.
 - Unabhängigkeit: Die Residuen sollten unabhängig voneinander sein.
 - Linearität: Der Zusammenhang zwischen X und Y sollte linear sein.

R-Erklärung:

- Die Funktion `predict()` berechnet vorhergesagte Werte für ein Regressionsmodell.
- Der Parameter `newdata` in `predict()` gibt einen Dataframe mit den Werten der Prädiktoren an, für die Vorhersagen gemacht werden sollen.
- Die Funktion `resid()` extrahiert die Residuen aus einem Regressionsmodell.
- Die Funktion `abline()` mit dem Parameter `h` fügt eine horizontale Linie zum Plot hinzu.

9 TSS und RSS

Wir untersuchen nun die Zerlegung der Gesamtvariation in erklärte und nicht erklärte Variation.

```
# Quadratsummen: TSS, RSS, ESS
TSS <- sum((sales - mean(sales))^2)
RSS <- sum((sales - lbf(TV))^2)
```

```

ESS <- sum((lbf(TV) - mean(lbf(TV)))^2)

# Alternative Berechnung von ESS
ESS_alt <- sum((lbf(TV) - mean(sales))^2)

# Ausgabe der Quadratsummen
cat("TSS (Totale Variation):", round(TSS, 2), "\n")

## TSS (Totale Variation): 5417.15

cat("RSS (Residuen):", round(RSS, 2), "\n")

## RSS (Residuen): 2102.53

cat("ESS (Erklärte Variation):", round(ESS, 2), "\n")

## ESS (Erklärte Variation): 3314.62

cat("ESS (Alternative Berechnung):", round(ESS_alt, 2), "\n")

## ESS (Alternative Berechnung): 3314.62

cat("TSS = ESS + RSS:", round(TSS, 2), "=", round(ESS, 2), "+", round(RSS, 2), "\n")

## TSS = ESS + RSS: 5417.15 = 3314.62 + 2102.53

cat("Differenz (sollte nahe 0 sein):", round(TSS - (ESS + RSS), 10), "\n")

## Differenz (sollte nahe 0 sein): 0

```

Statistische Erklärung:

- Die **Gesamtquadratsumme** (Total Sum of Squares, TSS) misst die Gesamtvariation in der abhängigen Variable: $TSS = \sum (y_i - \bar{y})^2$
- Die **Residuenquadratsumme** (Residual Sum of Squares, RSS) misst die nicht erklärte Variation: $RSS = \sum (y_i - \hat{y}_i)^2$
- Die **Erklärte Quadratsumme** (Explained Sum of Squares, ESS) misst die durch das Modell erklärte Variation: $ESS = \sum (\hat{y}_i - \bar{y})^2$
- Es gilt die Zerlegung: $TSS = ESS + RSS$
- Diese Zerlegung ist die Grundlage für das Bestimmtheitsmass R^2 .

R-Erklärung:

- Die Berechnung von TSS, RSS und ESS erfolgt direkt nach den entsprechenden Formeln.
- Die Funktion `mean()` berechnet den Mittelwert eines Vektors.
- Die Funktion `round()` rundet eine Zahl auf eine bestimmte Anzahl von Dezimalstellen.

10 Zerlegung der Variation

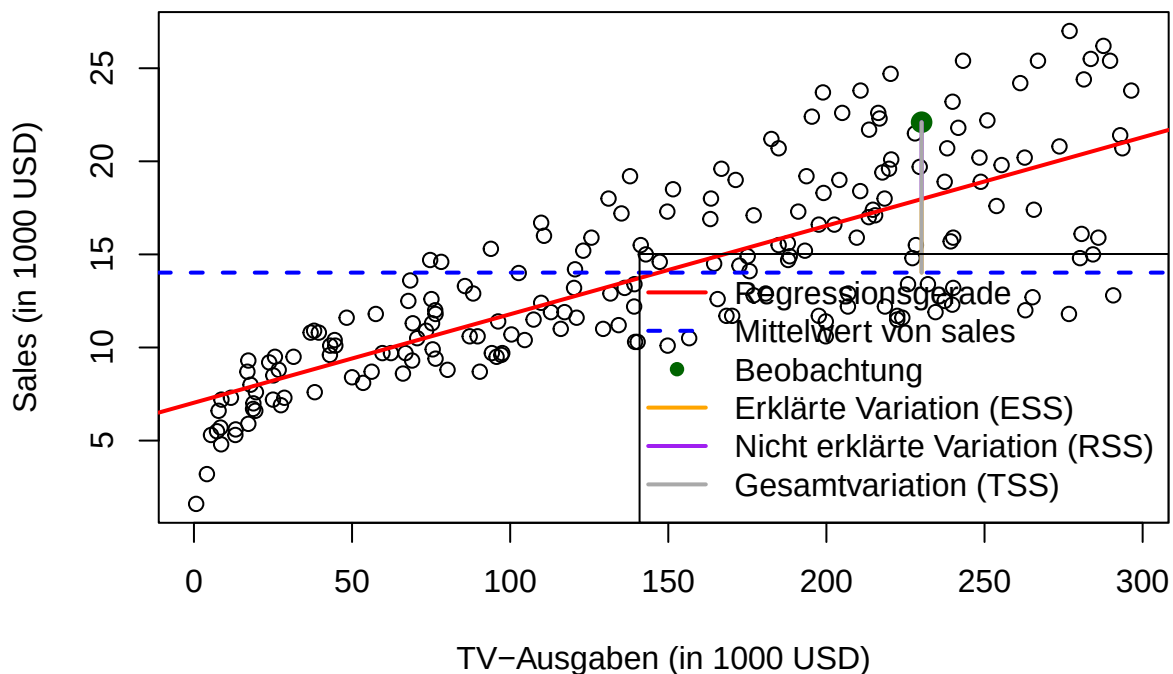
Wir visualisieren die Zerlegung der Gesamtvariation in erklärte und nicht erklärte Variation.

```
# Visualisierung der Zerlegung der Variation
plot(sales ~ TV,
     main = "Zerlegung der Variation",
     xlab = "TV-Ausgaben (in 1000 USD)",
     ylab = "Sales (in 1000 USD)")
abline(md1, col = "red", lwd = 2)
abline(h = mean(sales), col = "blue", lwd = 2, lty = 2)

# Für die erste Beobachtung
points(TV[1], sales[1], col = "darkgreen", pch = 16, cex = 1.5)
segments(TV[1], mean(sales), TV[1], pred_lbf[1], col = "orange", lwd = 2) # ESS
segments(TV[1], pred_lbf[1], TV[1], sales[1], col = "purple", lwd = 2) # RSS
segments(TV[1], mean(sales), TV[1], sales[1], col = "darkgray", lwd = 2) # TSS

# Legende hinzufügen
legend("bottomright",
      legend = c("Regressionsgerade", "Mittelwert von sales", "Beobachtung",
                 "Erklärte Variation (ESS)", "Nicht erklärte Variation (RSS)",
                 "Gesamtvariation (TSS)" ),
      col = c("red", "blue", "darkgreen", "orange", "purple", "darkgray"),
      lty = c(1, 2, NA, 1, 1, 1),
      pch = c(NA, NA, 16, NA, NA, NA),
      lwd = 2)
```

Zerlegung der Variation



```
# Ausgabe der Quadratsummen
cat("Zerlegung der Variation:\n")
```

```
## Zerlegung der Variation:
```



```
cat("TSS (Totale Variation):", round(TSS, 2), "\n")

## TSS (Totale Variation): 5417.15

cat("RSS (Residuen):", round(RSS, 2), "\n")

## RSS (Residuen): 2102.53

cat("ESS (Erklärte Variation):", round(ESS, 2), "\n")

## ESS (Erklärte Variation): 3314.62

cat("TSS = ESS + RSS:", round(TSS, 2), "=", round(ESS, 2), "+", round(RSS, 2), "\n")

## TSS = ESS + RSS: 5417.15 = 3314.62 + 2102.53
```

Statistische Erklärung:

- Die **Zerlegung der Variation** ist ein zentrales Konzept in der Regressionsanalyse:
 - Die Gesamtvariation (TSS) wird in erklärte Variation (ESS) und nicht erklärte Variation (RSS) zerlegt.
 - Die erklärte Variation (ESS) ist die Variation, die durch das Regressionsmodell erklärt wird.
 - Die nicht erklärte Variation (RSS) ist die Variation, die durch das Modell nicht erklärt wird (Residuen).
- Für jede Beobachtung gilt: $(y_i - \bar{y}) = (\hat{y}_i - \bar{y}) + (y_i - \hat{y}_i)$
 - $(y_i - \bar{y})$ ist die Gesamtabweichung (TSS-Komponente)
 - $(\hat{y}_i - \bar{y})$ ist die erklärte Abweichung (ESS-Komponente)
 - $(y_i - \hat{y}_i)$ ist die nicht erklärte Abweichung (RSS-Komponente)

R-Erklärung:

- Die Funktion `segments()` zeichnet Liniensegmente zwischen zwei Punkten.
- Die Parameter `col`, `lwd` und `lty` in `segments()` geben die Farbe, Breite und den Typ der Linie an.
- Die Funktion `points()` fügt Punkte zu einem bestehenden Plot hinzu.
- Die Funktion `legend()` fügt eine Legende zum Plot hinzu.

11 Bestimmtheitsmass

Das Bestimmtheitsmass R^2 gibt an, wie gut das Regressionsmodell die Variation in den Daten erklärt.

```
# Bestimmtheitsmass  $R^2$ 
r_squared <- (TSS - RSS) / TSS
r_squared_alt <- ESS / TSS
r_squared_check <- cor(sales, TV)^2

# Ausgabe der verschiedenen Berechnungen von  $R^2$ 
cat("Bestimmtheitsmass  $R^2$  (aus Varianzanalyse):", round(r_squared, 4), "\n")

## Bestimmtheitsmass  $R^2$  (aus Varianzanalyse): 0.6119

cat("Bestimmtheitsmass  $R^2$  (alternative Berechnung):", round(r_squared_alt, 4), "\n")

## Bestimmtheitsmass  $R^2$  (alternative Berechnung): 0.6119
```

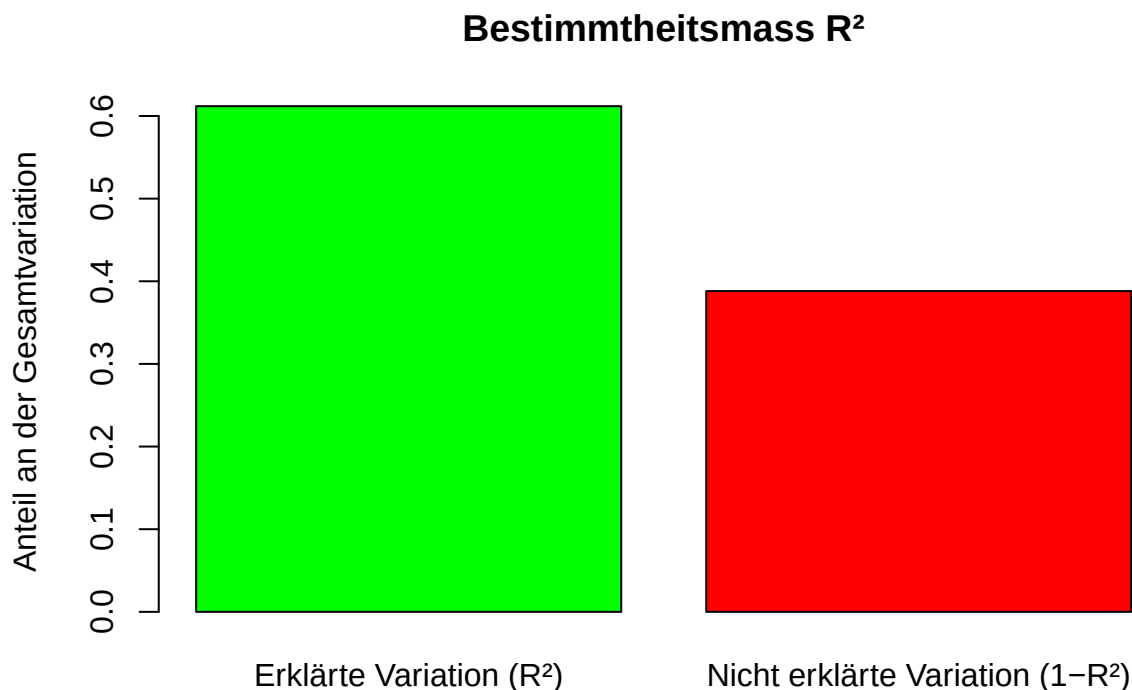
```
cat("Bestimmtheitsmass R2 (aus Korrelation):", round(r_squared_check, 4), "\n")

## Bestimmtheitsmass R2 (aus Korrelation): 0.6119

# R2 aus dem lm-Objekt
r_squared_lm <- summary(md1)$r.squared
cat("Bestimmtheitsmass R2 (aus lm()):", round(r_squared_lm, 4), "\n")

## Bestimmtheitsmass R2 (aus lm()): 0.6119

# Visualisierung von R2
barplot(c(r_squared, 1 - r_squared),
        names.arg = c("Erklärte Variation (R2)", "Nicht erklärte Variation (1-R2)"),
        col = c("green", "red"),
        main = "Bestimmtheitsmass R2",
        ylab = "Anteil an der Gesamtvariation")
```



Statistische Erklärung:

- Das **Bestimmtheitsmass R²** gibt den Anteil der Variation in der abhängigen Variable an, der durch das Regressionsmodell erklärt wird.
- R² liegt zwischen 0 und 1:
 - R² = 0: Das Modell erklärt keine Variation in den Daten.
 - R² = 1: Das Modell erklärt die gesamte Variation in den Daten.
- R² kann auf verschiedene Weisen berechnet werden:
 - R² = (TSS - RSS) / TSS
 - R² = ESS / TSS
 - R² = r², wobei r die Korrelation zwischen X und Y ist (nur für einfache lineare Regression)
- Ein hohes R² bedeutet, dass das Modell gut zu den Daten passt, aber es garantiert nicht, dass das Modell kausal oder prädiktiv ist.

- R^2 kann durch Hinzufügen weiterer Prädiktoren künstlich erhöht werden, daher wird in der multiplen Regression oft das adjustierte R^2 verwendet.

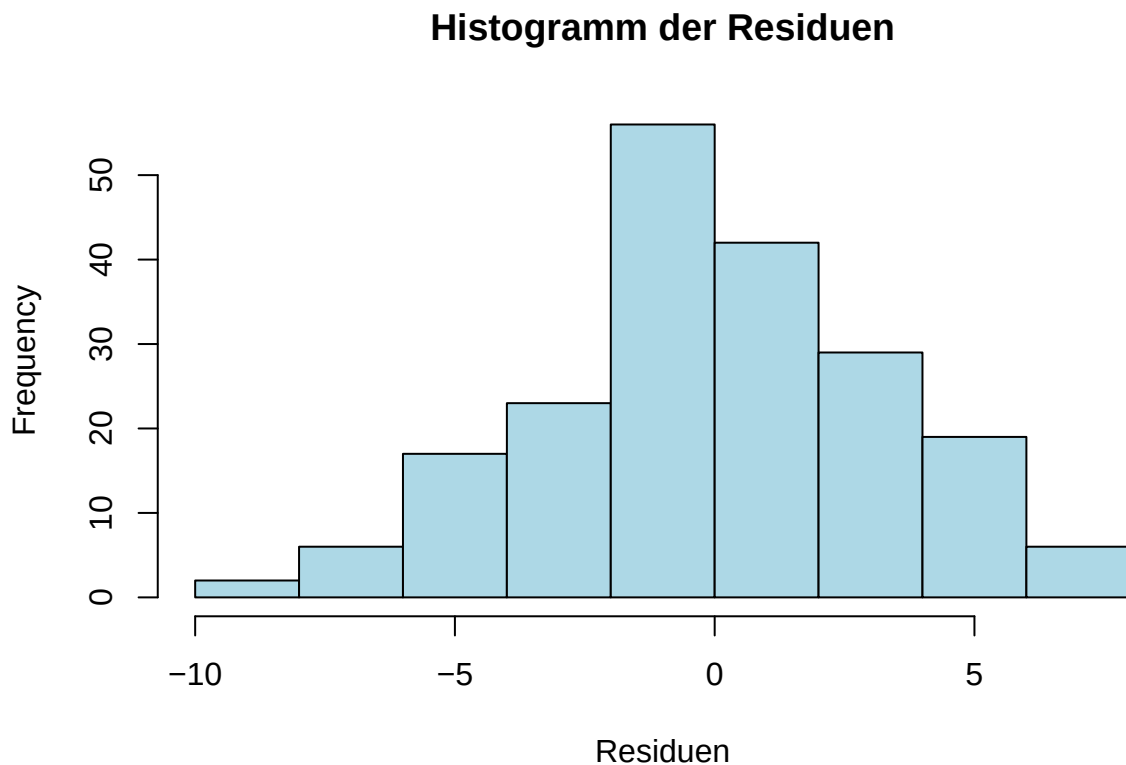
R-Erklärung:

- Die Funktion `summary()` liefert eine detaillierte Zusammenfassung des Regressionsmodells, einschliesslich R^2 .
- Der Ausdruck `summary(md1)$r.squared` extrahiert den R^2 -Wert aus dem Regressionsobjekt.
- Die Funktion `barplot()` erstellt ein Balkendiagramm.
- Der Parameter `names.arg` in `barplot()` gibt die Namen der Balken an.
- Der Parameter `col` in `barplot()` gibt die Farben der Balken an.

12 Erwartungen an Residuen

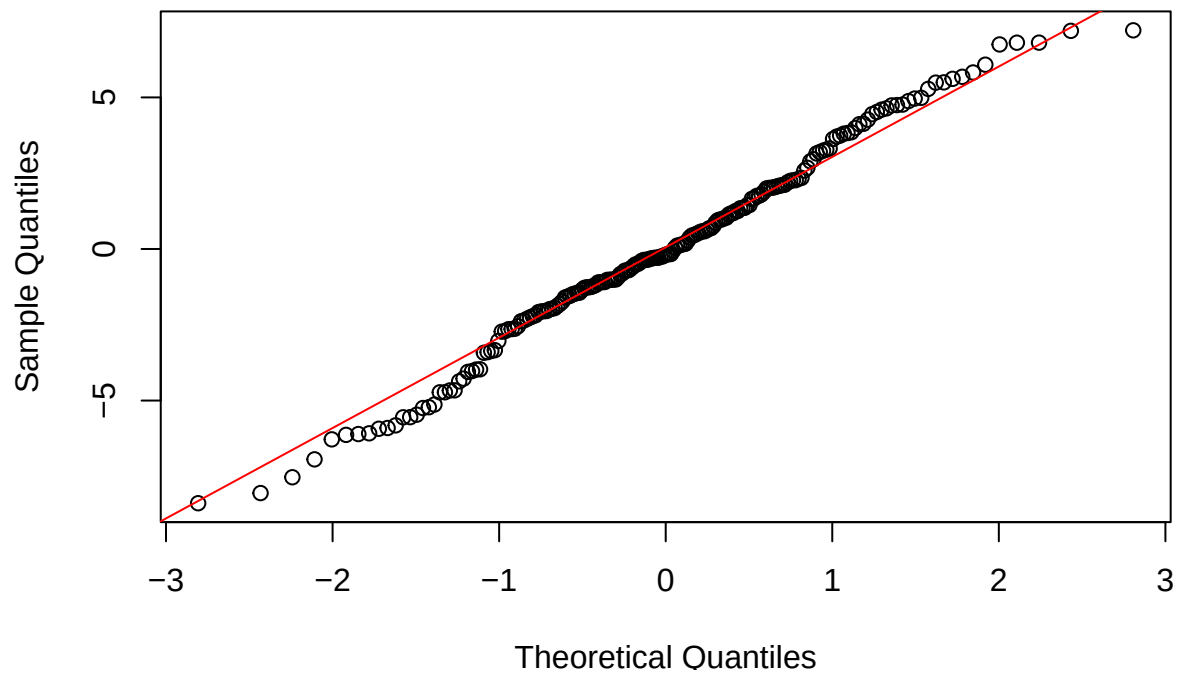
Wir untersuchen die Eigenschaften der Residuen, um die Annahmen der linearen Regression zu überprüfen.

```
# Residuen-Diagnostik
hist(resid(md1),
     main = "Histogramm der Residuen",
     xlab = "Residuen",
     col = "lightblue")
```

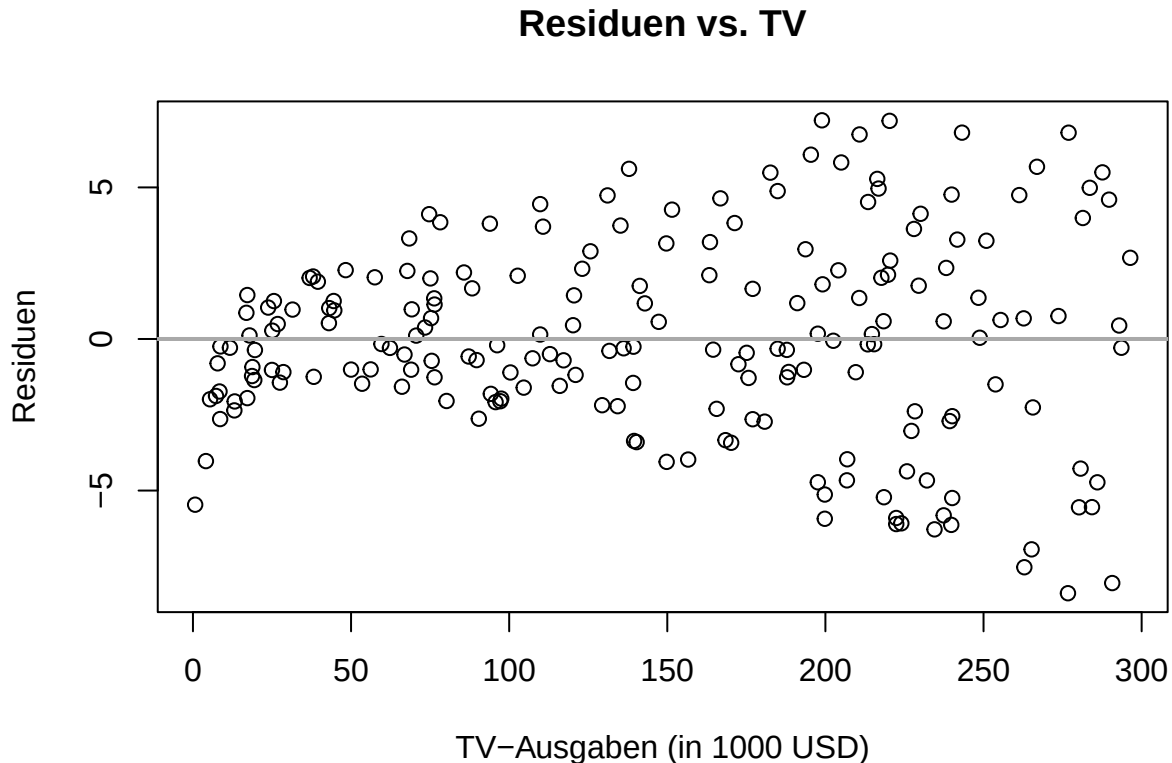


```
# QQ-Plot der Residuen
qqnorm(resid(md1), main = "Q-Q Plot der Residuen")
qqline(resid(md1), col = "red")
```

Q-Q Plot der Residuen



```
# Residuen vs. Prädiktor
plot(resid(md1) ~ TV,
     main = "Residuen vs. TV",
     xlab = "TV-Ausgaben (in 1000 USD)",
     ylab = "Residuen")
abline(h = 0, col = "darkgray", lwd = 2)
```



```
# Standardfehler der Regression (RSE)
n <- length(sales)
RSE_manual <- sqrt(RSS / (n - 2))
RSE_lm <- summary(md1)$sigma
cat("RSE (manuell):", round(RSE_manual, 3), "\n")
```

```
## RSE (manuell): 3.259
```

```
cat("RSE (aus lm()):", round(RSE_lm, 3), "\n")
```

```
## RSE (aus lm()): 3.259
```

```
# Interpretation des RSE
cat("Interpretation des RSE:\n")
```

```
## Interpretation des RSE:
```

```
cat("Der typische Fehler bei der Vorhersage von sales beträgt etwa", round(RSE_lm, 3),
    "\nTausend USD.\n")
```

```
## Der typische Fehler bei der Vorhersage von sales beträgt etwa 3.259 Tausend USD.
```

Statistische Erklärung:

- Die **Annahmen der linearen Regression** betreffen hauptsächlich die Eigenschaften der Residuen:
 - Normalität: Die Residuen sollten normalverteilt sein.
 - Homoskedastizität: Die Varianz der Residuen sollte konstant sein.
 - Unabhängigkeit: Die Residuen sollten unabhängig voneinander sein.

- Linearität: Der Zusammenhang zwischen X und Y sollte linear sein.
- Der **Standardfehler der Regression** (Residual Standard Error, RSE) ist ein Mass für die typische Grösse der Residuen:
 - $RSE = \sqrt{RSS / (n - 2)}$
 - Der RSE hat die gleiche Einheit wie die abhängige Variable.
 - Ein kleinerer RSE bedeutet eine bessere Anpassung des Modells an die Daten.
- Der RSE kann für die Konstruktion von Konfidenz- und Prognoseintervallen verwendet werden.

R-Erklärung:

- Die Funktion `qqnorm()` erstellt einen Q-Q-Plot, der die Verteilung der Residuen mit der Normalverteilung vergleicht.
- Die Funktion `qqline()` fügt eine Referenzlinie zum Q-Q-Plot hinzu.
- Der Ausdruck `summary(md1)$sigma` extrahiert den RSE aus dem Regressionsobjekt.
- Die Funktion `sqrt()` berechnet die Quadratwurzel einer Zahl.

13 Prognose

Schliesslich verwenden wir das Regressionsmodell, um Vorhersagen für neue Werte der unabhängigen Variable zu treffen.

```
# Prognose + Prognoseintervall
tv_new <- 230.1
predicted <- predict(md1, newdata = data.frame(TV = tv_new))
cat("Punktschätzung bei TV =", tv_new, ":", round(predicted, 2), "Tausend USD\n")
```

```
## Punktschätzung bei TV = 230.1 : 17.97 Tausend USD
```

```
# Approximatives 95%-Prognoseintervall
cat("95%-Prognoseintervall ",
    round(predicted - 2 * RSE_lm, 2), "bis",
    round(predicted + 2 * RSE_lm, 2), "Tausend USD\n")
```

```
## 95%-Prognoseintervall 11.45 bis 24.49 Tausend USD
```

```
# Exaktes Prognoseintervall
pred_interval <- predict(md1, newdata = data.frame(TV = tv_new), interval = "prediction",
  ↪ level = 0.95)
cat("Exaktes 95%-Prognoseintervall:",
    round(pred_interval[2], 2), "bis",
    round(pred_interval[3], 2), "Tausend USD\n")
```

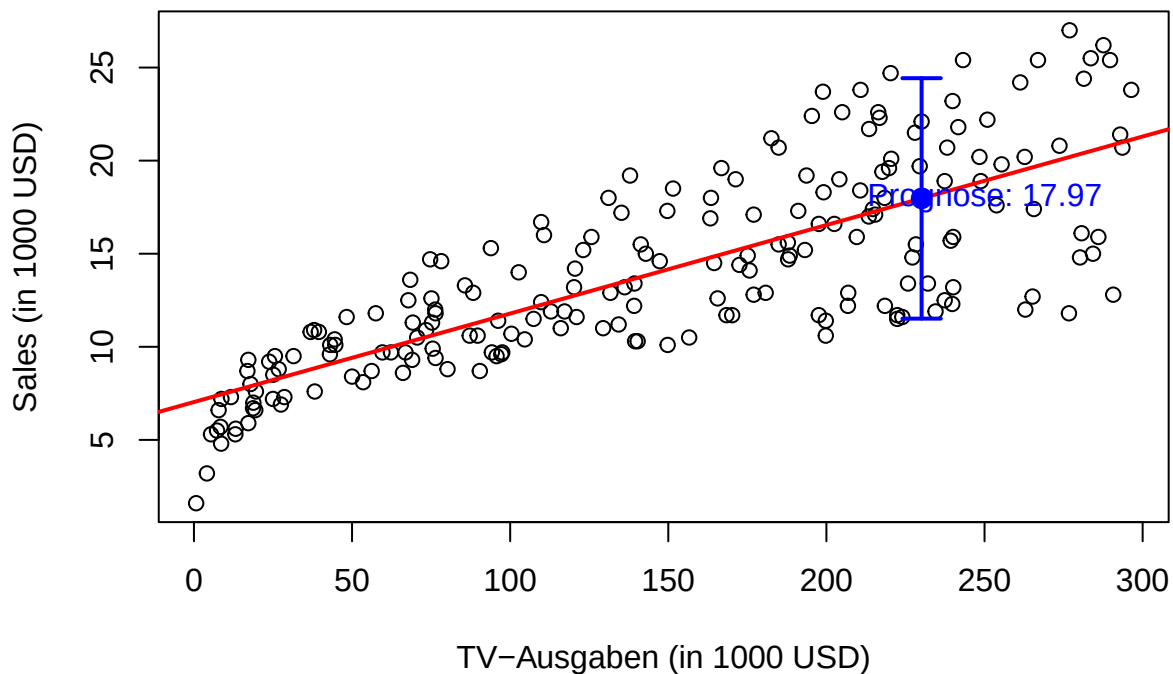
```
## Exaktes 95%-Prognoseintervall: 11.51 bis 24.43 Tausend USD
```

```
# Konfidenzintervall für den Erwartungswert
conf_interval <- predict(md1, newdata = data.frame(TV = tv_new), interval = "confidence",
  ↪ level = 0.95)
cat("95%-Konfidenzintervall für den Erwartungswert:",
    round(conf_interval[2], 2), "bis",
    round(conf_interval[3], 2), "Tausend USD\n")
```

```
## 95%-Konfidenzintervall für den Erwartungswert: 17.34 bis 18.6 Tausend USD
```

```
# Visualisierung der Prognose
plot(sales ~ TV,
     main = "Prognose bei TV = 230.1",
     xlab = "TV-Ausgaben (in 1000 USD)",
     ylab = "Sales (in 1000 USD)")
abline(md1, col = "red", lwd = 2)
points(tv_new, predicted, col = "blue", pch = 16, cex = 1.5)
arrows(tv_new, pred_interval[2], tv_new, pred_interval[3],
       code = 3, angle = 90, length = 0.1, col = "blue", lwd = 2)
text(tv_new + 20, predicted,
     paste("Prognose:", round(predicted, 2)),
     col = "blue")
```

Prognose bei TV = 230.1



```
# Interpretation der Koeffizienten
cat("\nInterpretation der Koeffizienten:\n")
```

```
##
## Interpretation der Koeffizienten:
```

```
cat("- Achsenabschnitt:", round(coef(md1)[1], 2),
     "\n-> erwartete sales bei TV = 0 (in Tausend USD)\n")
```

```
## - Achsenabschnitt: 7.03 -> erwartete sales bei TV = 0 (in Tausend USD)
```

```
cat("- Steigung:", round(coef(md1)[2], 4),
     "\n-> erwarteter Anstieg von sales bei +1 Einheit TV (in Tausend USD)\n")
```

```
## - Steigung: 0.0475 → erwarteter Anstieg von sales bei +1 Einheit TV (in Tausend USD)
```

```
cat(" (d.h. für jeden zusätzlichen Tausend USD an TV-Werbung\n")
```

```
## (d.h. für jeden zusätzlichen Tausend USD an TV-Werbung
```

```
cat(" steigen die erwarteten Verkäufe um ca.", round(coef(md1)[2] * 1000, 2), "USD)\n")
```

```
## steigen die erwarteten Verkäufe um ca. 47.54 USD)
```

Statistische Erklärung:

- Die **Prognose** ist der vorhergesagte Wert der abhängigen Variable für einen bestimmten Wert der unabhängigen Variable: $\hat{y} = b + b x$
- Das **Prognoseintervall** gibt den Bereich an, in dem ein einzelner neuer Wert mit einer bestimmten Wahrscheinlichkeit liegen wird.
- Das **Konfidenzintervall für den Erwartungswert** gibt den Bereich an, in dem der wahre Erwartungswert mit einer bestimmten Wahrscheinlichkeit liegt.
- Prognoseintervalle sind breiter als Konfidenzintervalle für den Erwartungswert, da sie zusätzlich die Variation der einzelnen Beobachtungen um den Erwartungswert berücksichtigen.
- Die **Interpretation der Koeffizienten** ist ein wichtiger Aspekt der Regressionsanalyse:
 - Der Achsenabschnitt (b) ist der erwartete Wert der abhängigen Variable, wenn die unabhängige Variable Null ist.
 - Die Steigung (b) gibt an, um wie viel sich die abhängige Variable im Durchschnitt ändert, wenn die unabhängige Variable um eine Einheit zunimmt.

R-Erklärung:

- Die Funktion `predict()` mit dem Parameter `interval = "prediction"` berechnet Prognoseintervalle.
- Die Funktion `predict()` mit dem Parameter `interval = "confidence"` berechnet Konfidenzintervalle für den Erwartungswert.
- Der Parameter `level` in `predict()` gibt das Konfidenzniveau an.
- Die Funktion `arrows()` zeichnet Pfeile zwischen zwei Punkten.
- Die Funktion `text()` fügt Text zu einem Plot hinzu.

14 Zusammenfassung

In dieser Vorlesung haben wir die einfache lineare Regression kennengelernt, ein grundlegendes statistisches Verfahren zur Modellierung des Zusammenhangs zwischen zwei Variablen. Die wichtigsten Konzepte waren:

1. **Variation und Kovarianz:** Die Variation in den Daten und der Zusammenhang zwischen den Variablen bilden die Grundlage für die Regression.
2. **Methode der kleinsten Quadrate:** Diese Methode findet die "beste" Regressionsgerade, indem sie die Summe der quadrierten Residuen minimiert.
3. **Residuen:** Die Abweichungen zwischen den beobachteten und den vorhergesagten Werten geben Aufschluss über die Qualität der Anpassung.
4. **Zerlegung der Variation:** Die Gesamtvariation wird in erklärte und nicht erklärte Variation zerlegt.
5. **Bestimmtheitsmass (R^2):** Dieses Mass gibt an, wie gut das Modell die Variation in den Daten erklärt.
6. **Prognose:** Das Regressionsmodell kann für Vorhersagen verwendet werden, wobei Prognoseintervalle die Unsicherheit quantifizieren.

Die einfache lineare Regression ist ein mächtiges Werkzeug für die Datenanalyse, hat aber auch Einschränkungen. Sie setzt einen linearen Zusammenhang zwischen den Variablen voraus und kann komplexere Beziehungen nicht erfassen. In solchen Fällen können fortgeschrittenere Methoden wie multiple Regression, polynomiale Regression oder nicht-parametrische Verfahren angemessener sein.